

# Algorithms Reference

Flowcharts for the five classic algorithms:

- › Linear Search
- › Binary Search
- › Bubble Sort
- › Insertion Sort
- › Selection Sort

---

Each section includes a flowchart with explanatory comments and a complexity reference.

# Contents

|    |                    |        |
|----|--------------------|--------|
| 1. | Linear Search      | page 3 |
| 2. | Binary Search      | page 4 |
| 3. | Bubble Sort        | page 5 |
| 4. | Insertion Sort     | page 6 |
| 5. | Selection Sort     | page 7 |
|    | Complexity Summary | page 8 |

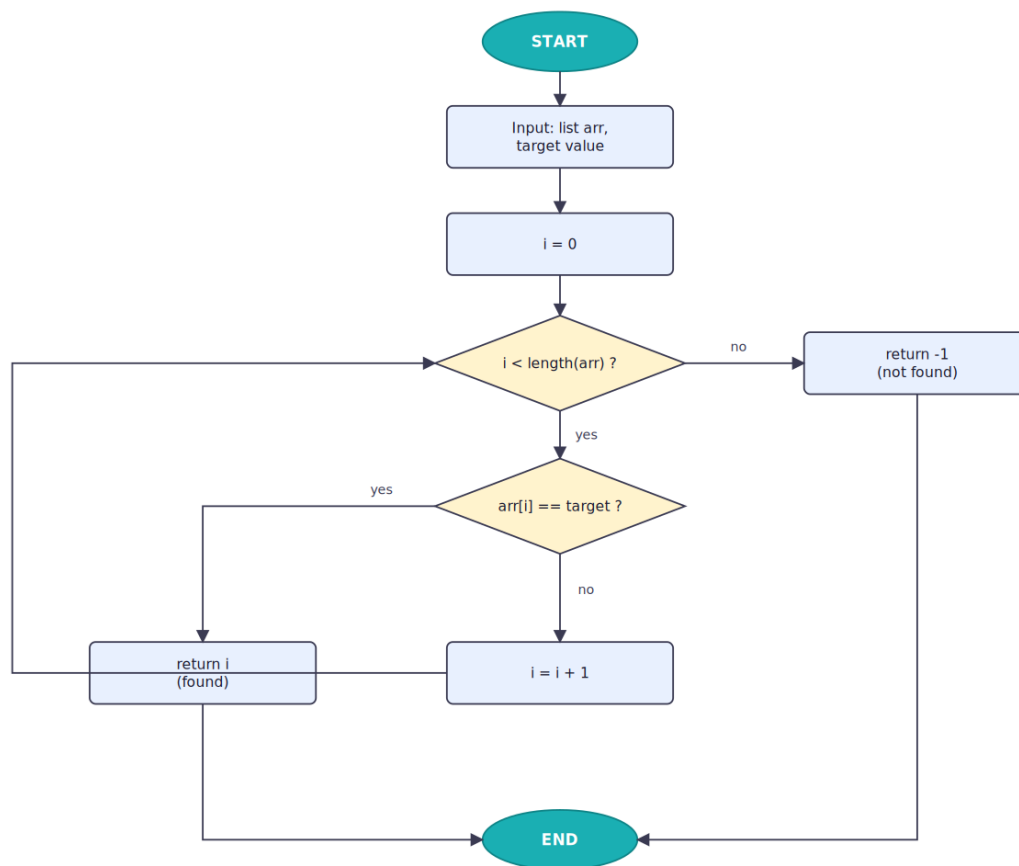
**Legend.** In every flowchart, ovals mark **start** and **end**; rectangles mark **actions**; and diamonds mark **decisions**. Arrows show the direction of flow; *yes/no* labels mark which branch a decision takes.

# 1. Linear Search

**Idea.** Walk through the list one element at a time, comparing each item with the target. Return the index of the first match, or  $-1$  if the target is not in the list.

**When to use.** The list is small, unsorted, or used only once. Linear search makes no assumptions about ordering.

## Flowchart



*All paths flow top-to-bottom and converge at a single END.*

## Complexity

| Algorithm     | Best   | Average | Worst  | Space  |
|---------------|--------|---------|--------|--------|
| Linear Search | $O(1)$ | $O(n)$  | $O(n)$ | $O(1)$ |

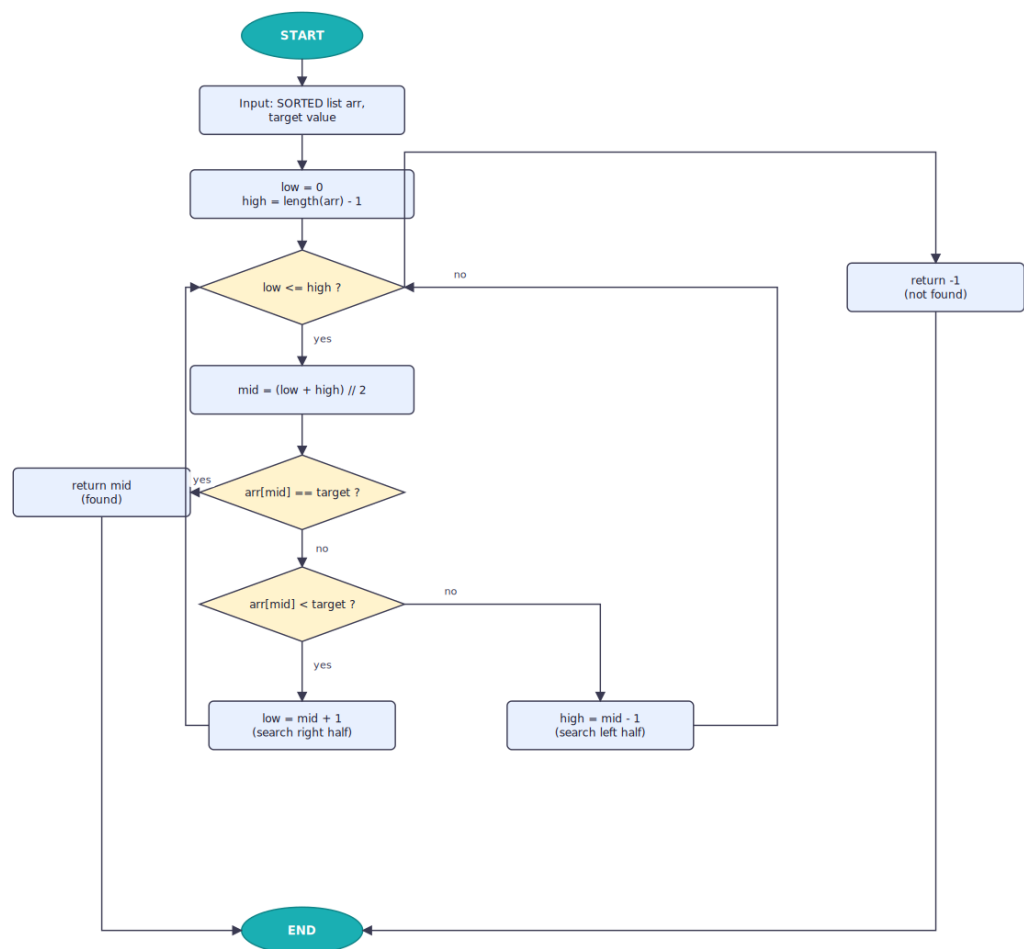
*Best case:* the target is at index 0. *Worst case:* the target is missing or at the last position.

## 2. Binary Search

**Idea.** The list **must already be sorted**. Compare the target with the middle element. If they match, done. If the target is smaller, repeat on the left half; otherwise on the right half. Each step throws away half of the remaining items.

**When to use.** Large sorted lists. Binary search reaches a billion-item list in around 30 comparisons.

### Flowchart



### Complexity

| Algorithm     | Best   | Average     | Worst       | Space  |
|---------------|--------|-------------|-------------|--------|
| Binary Search | $O(1)$ | $O(\log n)$ | $O(\log n)$ | $O(1)$ |

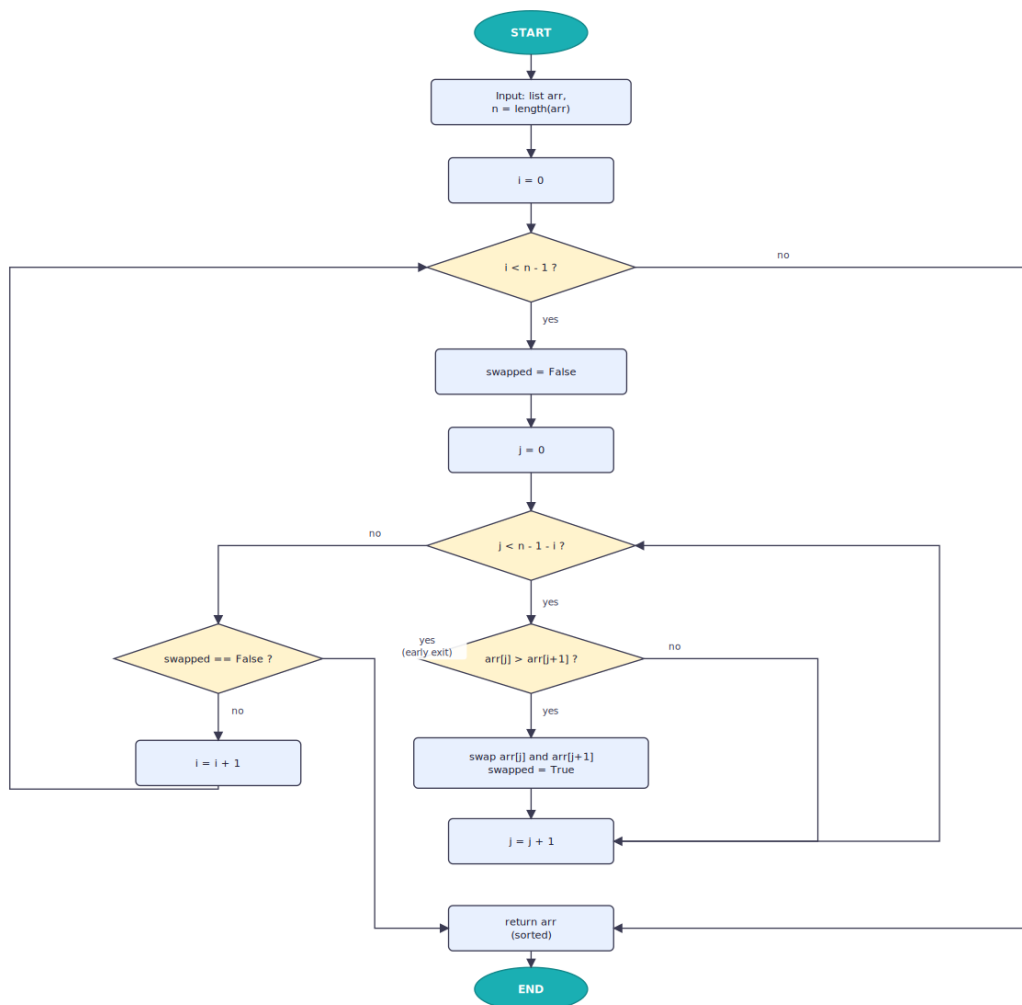
*Note:* Binary search itself is  $O(\log n)$ , but if you have to sort the list first the total cost is dominated by the sort.

## 3. Bubble Sort

**Idea.** Walk through the list comparing adjacent pairs and swapping any that are out of order. Each full pass ‘bubbles’ the largest unsorted value to the end. Repeat until no swaps are needed.

**When to use.** Mostly for teaching the concept of comparison and swapping. In production, prefer faster algorithms.

### Flowchart



### Complexity

| Algorithm   | Best   | Average  | Worst    | Space  |
|-------------|--------|----------|----------|--------|
| Bubble Sort | $O(n)$ | $O(n^2)$ | $O(n^2)$ | $O(1)$ |

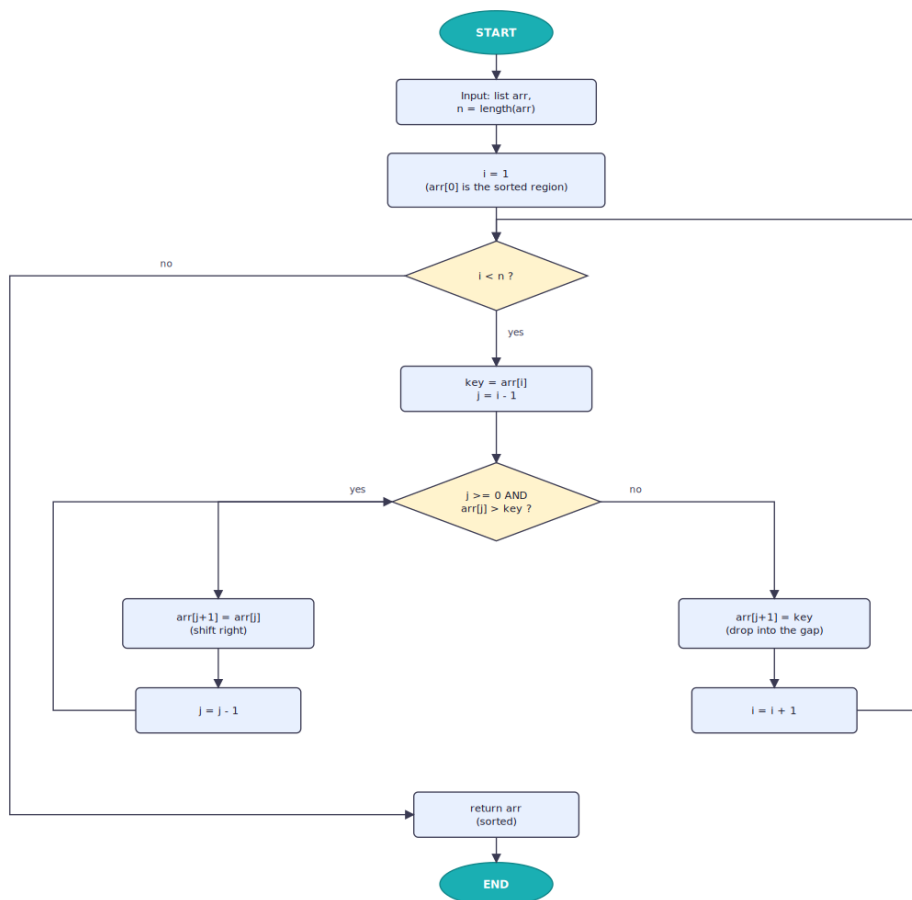
Best case  $O(n)$  only with the early-exit optimization on an already-sorted input. Without it, bubble sort is  $O(n^2)$  in every case.

## 4. Insertion Sort

**Idea.** Treat the first element as a sorted region of size 1. Take each next value (the 'key') and walk left through the sorted region, shifting larger values one slot to the right until the key's correct place is found. Drop it into the gap.

**When to use.** Small lists, or lists that are already nearly sorted. Insertion sort is very fast in those cases.

### Flowchart



### Complexity

| Algorithm      | Best   | Average  | Worst    | Space  |
|----------------|--------|----------|----------|--------|
| Insertion Sort | $O(n)$ | $O(n^2)$ | $O(n^2)$ | $O(1)$ |

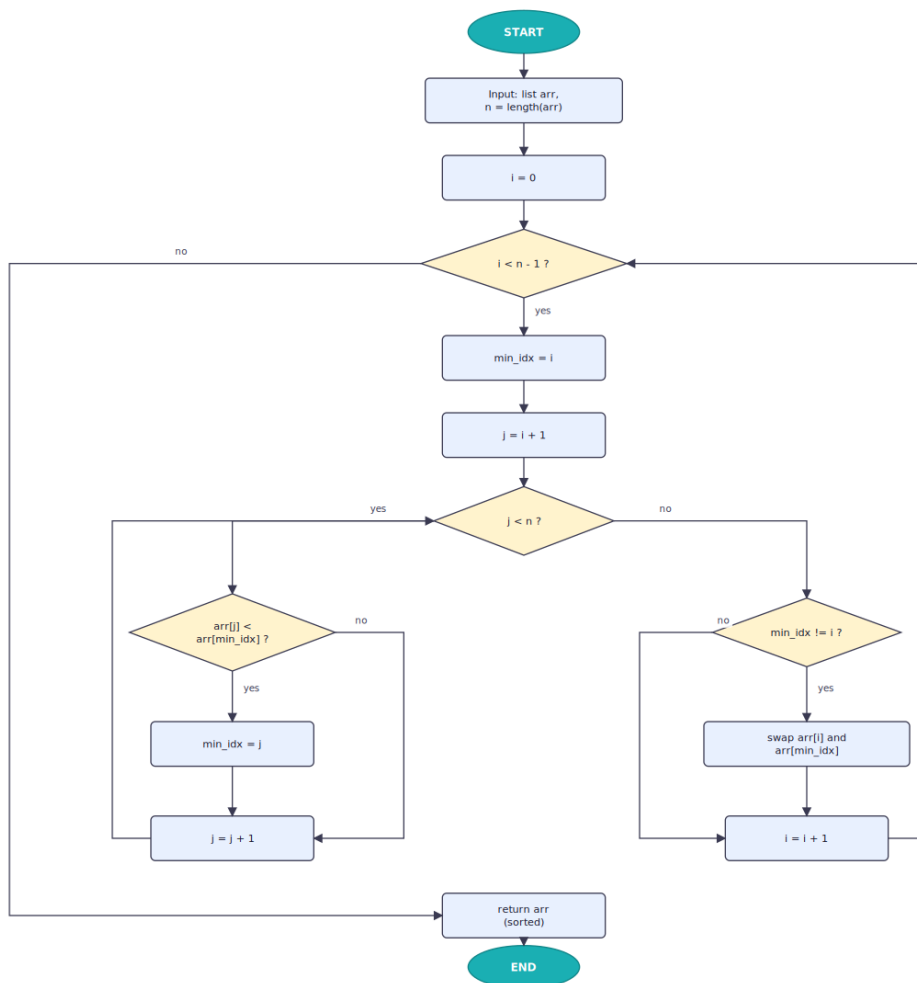
Best case  $O(n)$  when the array is already sorted: the inner loop exits immediately on every iteration.

## 5. Selection Sort

**Idea.** On each pass, scan the unsorted region to find the smallest value, then swap it with the first unsorted position. After each pass the sorted region grows by one and is locked in.

**When to use.** When write operations are expensive: selection sort performs at most  $n-1$  swaps in total, the fewest of any comparison-based sort.

### Flowchart



### Complexity

| Algorithm      | Best     | Average  | Worst    | Space  |
|----------------|----------|----------|----------|--------|
| Selection Sort | $O(n^2)$ | $O(n^2)$ | $O(n^2)$ | $O(1)$ |

Always  $O(n^2)$  comparisons regardless of input order, but at most  $n-1$  swaps total. Predictable but does not benefit from already-sorted data.

# Complexity Summary

---

Side-by-side comparison of all five algorithms.

| Algorithm      | Best     | Average     | Worst       | Space  |
|----------------|----------|-------------|-------------|--------|
| Linear Search  | $O(1)$   | $O(n)$      | $O(n)$      | $O(1)$ |
| Binary Search  | $O(1)$   | $O(\log n)$ | $O(\log n)$ | $O(1)$ |
| Bubble Sort    | $O(n)$   | $O(n^2)$    | $O(n^2)$    | $O(1)$ |
| Insertion Sort | $O(n)$   | $O(n^2)$    | $O(n^2)$    | $O(1)$ |
| Selection Sort | $O(n^2)$ | $O(n^2)$    | $O(n^2)$    | $O(1)$ |

## Practical guidance

- › **Searching.** Use **linear search** for unsorted or small lists. Use **binary search** when the list is sorted and you will look up many values.
- › **Sorting small lists.** **Insertion sort** is the fastest of these three on small or nearly-sorted inputs. **Bubble sort** with the early-exit optimization behaves similarly but is slightly slower in practice.
- › **Minimising writes.** **Selection sort** performs at most  $n-1$  swaps total, making it attractive when each write is expensive (for example, writing to flash memory).
- › **Production use.** All three sorts here are  $O(n^2)$ . Real software uses  $O(n \log n)$  algorithms such as merge sort, quicksort, or Timsort (Python's built-in `sorted()`).